

Lab8

402102562 - 402101758

AMITIS MIRABEDINI – NEGIN RAKI

Q1:

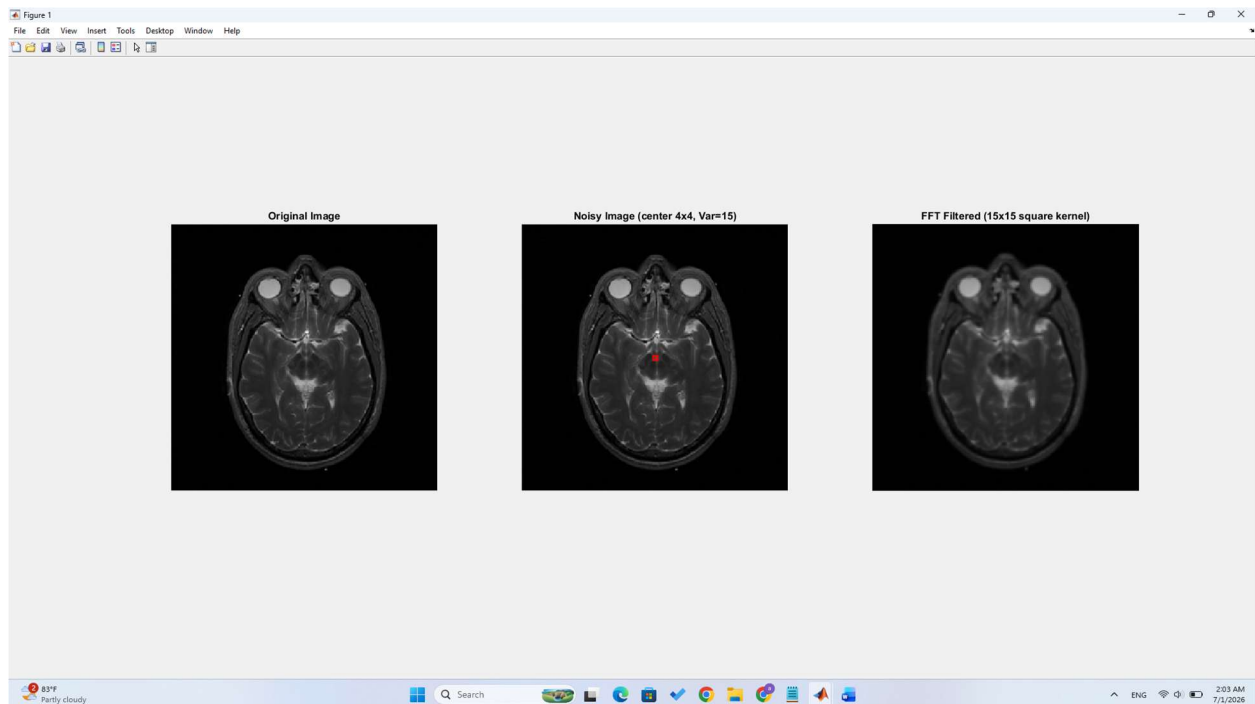
Using the script below, we can obtain the original and noisy image. Next, we construct a normalized square kernel of size 4x4, transform it to the frequency domain, and multiply it by the Fourier transform of the noisy image. Finally, we convert the result back to the spatial domain. The final result is shown in the figure below.

```
1 %% Q1
2 clear all;
3 close all;
4 clc;
5 addpath('E:\6th Semester\MISP Lab\MyLab\LAB8\Lab 8_data\S2_Q1_utils');
6 img = imread("E:\6th Semester\MISP Lab\MyLab\LAB8\Lab 8_data\S2_Q1_utils\t2.jpg");
7 first_slice = double(img(:,1));
8 % Add Gaussian noise to center 4x4 region (not entire image)
9 [M, N] = size(first_slice);
10 noise_block = sqrt(15) * randn(4, 4);
11
12 r0 = floor((M-4)/2) + 1;
13 c0 = floor((N-4)/2) + 1;
14
15
16 noisy_first_slice = first_slice;
17 noisy_first_slice(r0:r0+3, c0:c0+3) = noisy_first_slice(r0:r0+3, c0:c0+3) + noise_block;
18 noisy_first_slice = uint8(max(min(noisy_first_slice, 255), 0));
19
20 % Create square kernel of size 15 with variance 1
21 kernel_size = 15;
22 variance = 1;
23
24 [x, y] = meshgrid(-(kernel_size-1)/2:(kernel_size-1)/2, ...
25                 -(kernel_size-1)/2:(kernel_size-1)/2);
26 square_kernel = exp(-(x.^2 + y.^2) / (2 * variance));
27
28
29 square_kernel = square_kernel / sum(square_kernel(:));
30
31 % Fourier domain filtering with square kernel
32
33 kernel_padded = zeros(M, N);
34 start_row = floor((M - kernel_size)/2) + 1;
35 start_col = floor((N - kernel_size)/2) + 1;
36 kernel_padded(start_row:start_row+kernel_size-1, start_col:start_col+kernel_size-1) = square_kernel;
37
38 kernel_shifted = fftshift(kernel_padded);
39
40 fft_first = fft2(first_slice);
41 fft_kernel = fft2(kernel_shifted);
42 filtered_fft = fft_first .* fft_kernel;
43 filtered_slice = real(ifft2(filtered_fft));
44 filtered_disp = uint8(max(min(filtered_slice, 255), 0));
45
46 % Spatial domain filtering with Gaussian kernel
47 sigma = 1;
48 gauss_filtered = imgaussfilt(first_slice, sigma);
49 gauss_disp = uint8(max(min(gauss_filtered, 255), 0));
```

```

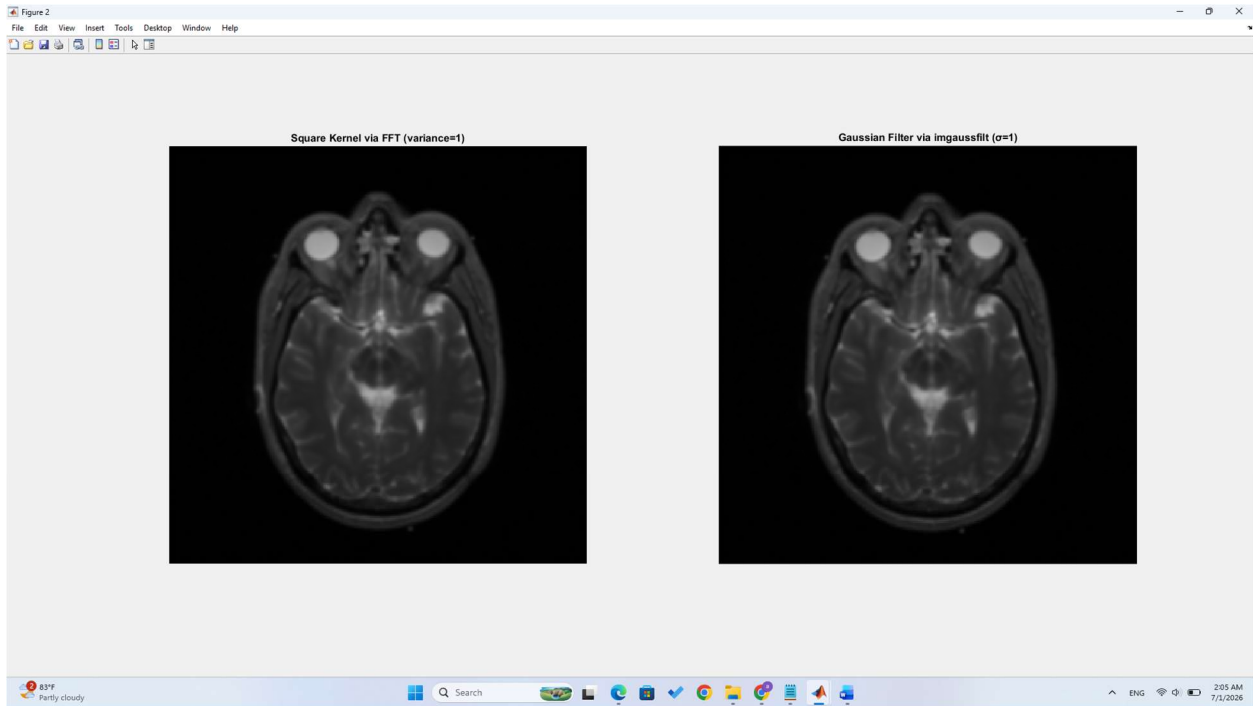
50 figure('Position', [100, 100, 1200, 400]);
51 subplot(1, 3, 1);
52 imshow(uint8(first_slice));
53 title('Original Image', 'FontSize', 12);
54
55 subplot(1, 3, 2);
56 imshow(noisy_first_slice);
57 title('Noisy Image (center 4x4, Var=15)', 'FontSize', 12);
58 hold on;
59 rectangle('Position', [c0, r0, 4, 4], 'EdgeColor', 'r', 'LineWidth', 2);
60 hold off;
61
62 subplot(1, 3, 3);
63 imshow(filtered_disp);
64 title('FFT Filtered (15x15 square kernel)', 'FontSize', 12);
65
66 figure('Position', [100, 100, 1000, 400]);
67
68 subplot(1, 2, 1);
69 imshow(filtered_disp);
70 title('Square Kernel via FFT (variance=1)', 'FontSize', 12);
71
72 subplot(1, 2, 2);
73 imshow(gauss_disp);
74 title('Gaussian Filter via imgaussfilt (sigma=1)', 'FontSize', 12);
75
76
77 figure('Position', [100, 100, 1200, 400]);
78
79 subplot(1, 3, 1);
80 diff_image = abs(double(filtered_disp) - double(gauss_disp));
81 imshow(diff_image, []);
82 title('Difference (FFT - Spatial)', 'FontSize', 12);
83 colorbar;
84
85 subplot(1, 3, 2);
86 mesh(x, y, square_kernel);
87 title('15x15 Square Kernel (variance=1)', 'FontSize', 12);
88 xlabel('x'); ylabel('y'); zlabel('Weight');
89
90 subplot(1, 3, 3);
91 kernel_fft_show = fftshift(abs(fft2(square_kernel, 256, 256)));
92 imagesc(kernel_fft_show);
93 title('Frequency Response of Kernel', 'FontSize', 12);
94 colorbar;
95 axis square;
96
97

```



By comparing them we can conclude that most of the gaussian noise is gone and the image quality is better now.

Now we use `imgaussfilt` to plot the results below

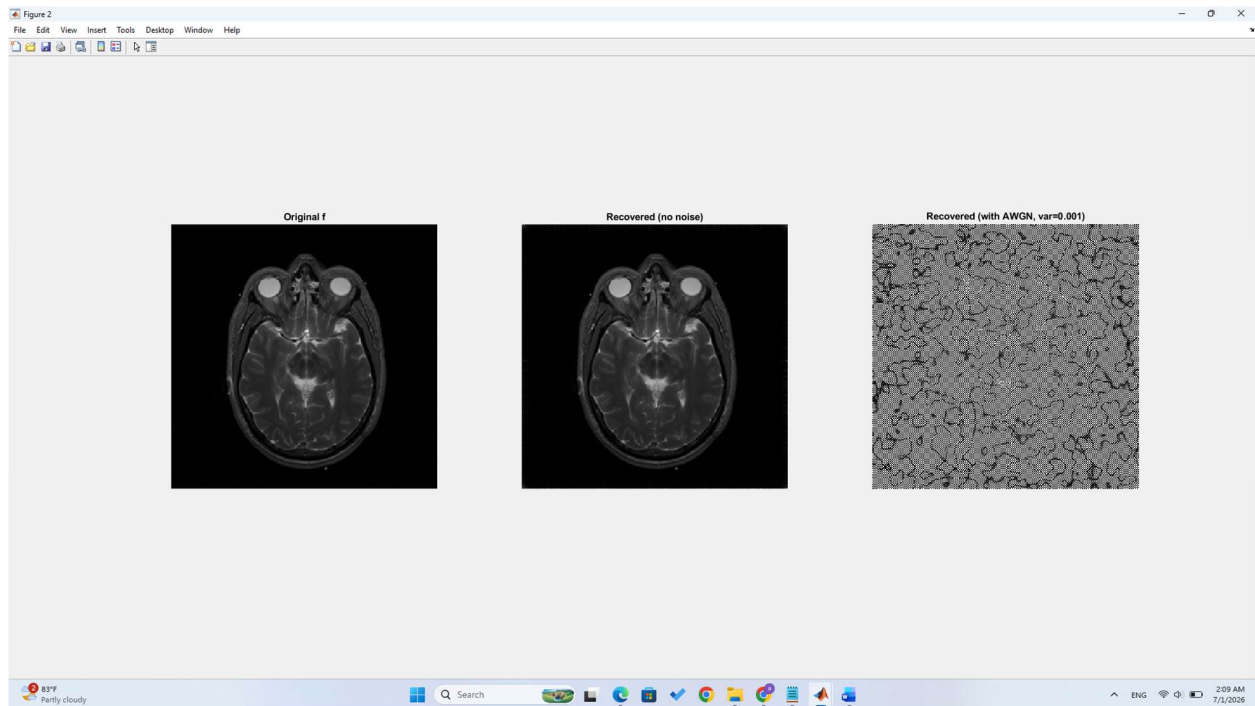
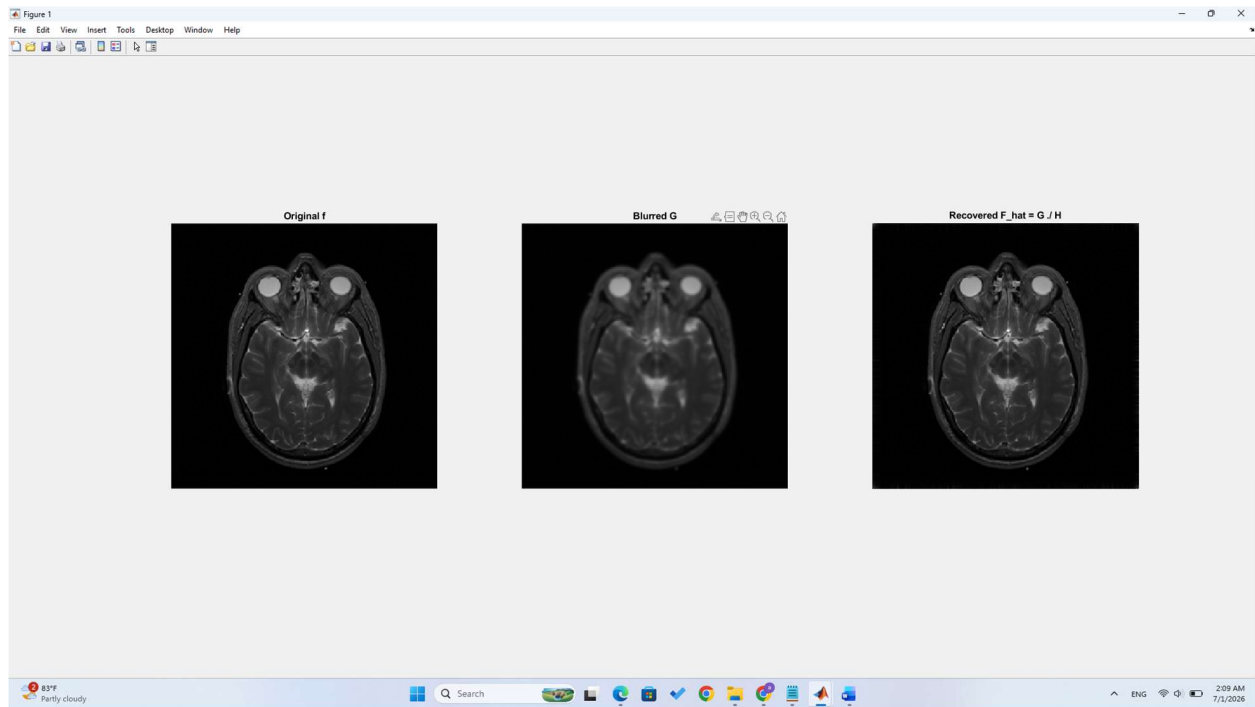


We can see that the gaussian noise has been removed here as well quite good and the result is almost the same as the applying the 4×4 square kernel but it has more sharpness and is less blurry.

Q2:

Using the script below we plot the original, blurred (+noisy) and the reconstructed channel as shown below.

```
98 %% Q2
99 clc;
100 addpath('E:\6th Semester\MISP Lab\MyLab\LAB8\Lab 8_data\S2_Q2_utils');
101 img = imread('E:\6th Semester\MISP Lab\MyLab\LAB8\Lab 8_data\S2_Q2_utils\t2.jpg');
102 first_slice = double(img(:,:,1));
103 [M, N] = size(first_slice);
104 h = Gaussian(1.2,[256,256]);
105 H = fft2(h);
106 G = conv2(first_slice, h, 'same');
107 F_hat = fftshift(real(ifft2(fft2(G) ./ H)));
108
109 G_disp = uint8( max(min(G, 255), 0) );
110 Fhat_disp = uint8( max(min(F_hat,255), 0) );
111 orig_disp = uint8( max(min(first_slice, 255), 0) );
112
113 figure;
114 subplot(1,3,1);
115 imshow(orig_disp);
116 title('Original f');
117
118 subplot(1,3,2);
119 imshow(G_disp);
120 title(['Blurred G']);
121
122 subplot(1,3,3);
123 imshow(Fhat_disp);
124 title('Recovered F\hat = G ./ H');
125
126 G_noisy = G + sqrt(0.001) * randn(size(G));
127 F_hat_noisy = fftshift(real(ifft2( fft2(G_noisy) ./ H )));
128 Fhat_noisy_disp = uint8( max(min(F_hat_noisy,255),0) );
129
130 figure;
131 subplot(1,3,1);
132 imshow(orig_disp);
133 title('Original f');
134
135 subplot(1,3,2);
136 imshow(Fhat_disp);
137 title('Recovered (no noise)');
138
139 subplot(1,3,3);
140 imshow(Fhat_noisy_disp);
141 title('Recovered (with AWGN, var=0.001)');
```



As we can observe applying ifft directly on the noisy image doesn't reconstruct the image in a meaningful way.

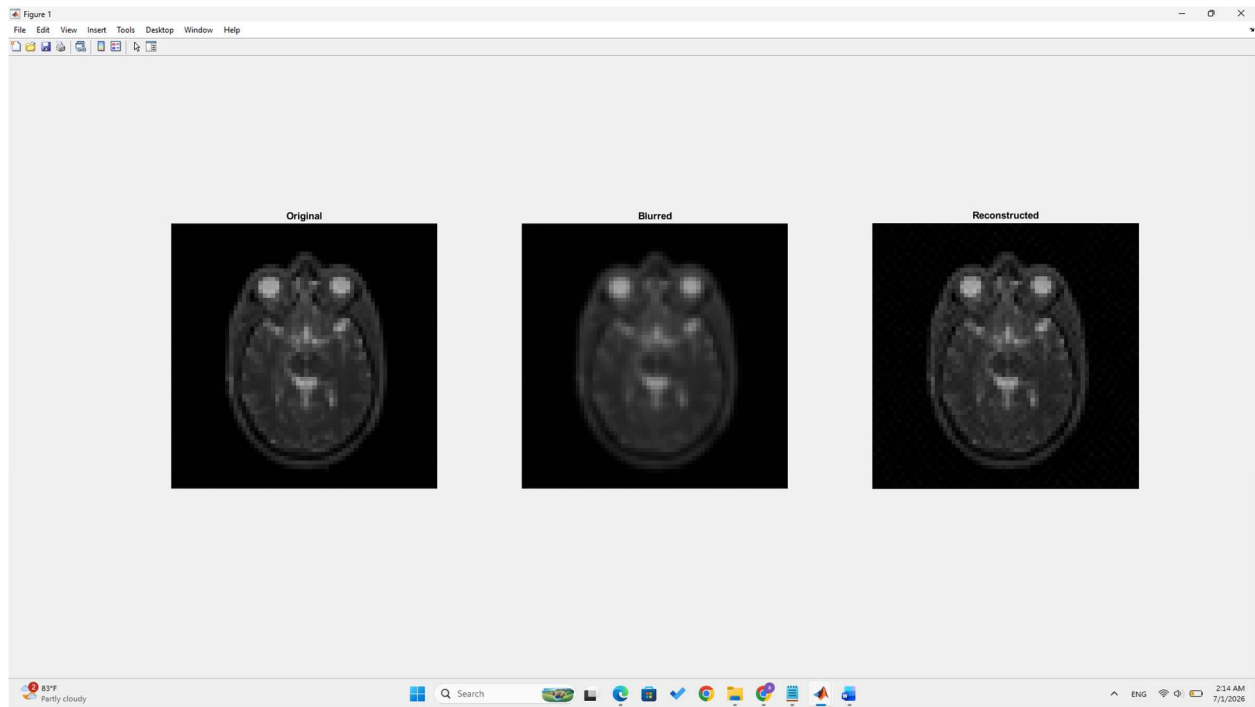
Q3:

Using the script below, we obtain the required results and inspect them as below:

```
143 %% Q3
144 clc; clear;
145 img = imread("E:\6th Semester\MISP Lab\MyLab\LAB8\Lab 8_data\S2_Q2_utils\t2.jpg");
146 f = double(img(:,:,1));
147 f = imresize(f, [64 64]);
148 N = 64;
149
150 h = [0 1 0;
151     1 2 1;
152     0 1 0];
153
154 h = h / sum(h(:)); % optional normalization
155
156 f_vec = f(:);
157
158 D = zeros(N*N, N*N);
159
160 for i = 1:N
161     for j = 1:N
162
163         % impulse image at (i,j)
164         delta = zeros(N,N);
165         delta(i,j) = 1;
166
167         % circular convolution with kernel
168         conv_result = conv2(delta, h, 'same');
169
170         % vectorize and assign as column
171         col_index = (j-1)*N + i;
172         D(:, col_index) = conv_result(:);
173     end
174 end
175
176 g = D * f_vec;
177 g_noisy = g + 0.05 * randn(size(g));
178 f_hat = pinv(D) * g_noisy;
179 f_hat_img = reshape(f_hat, N, N);
180 figure;
181 subplot(1,3,1);
182 imshow(uint8(f));
183 title('Original');
184 subplot(1,3,2);
185 imshow(uint8(reshape(g, N, N)));
186 title('Blurred');
187 subplot(1,3,3);
188 imshow(uint8(f_hat_img));
189 title('Reconstructed');
```

Why Does the Matrix (D) Model Convolution?

Convolution is a linear operation. The response of a linear system to an image can therefore be represented as the weighted sum of its responses to individual impulse pixels. Each row (or column) of the matrix (D) corresponds to the blurred response produced by a single pixel located at a specific position in the image. Circularly shifting the kernel generates the impulse response for every possible pixel location. As a result, multiplying (D) by the vectorized image performs exactly the same operation as applying the blur filter to the image through convolution.



Results and Discussion

The blurred image exhibits a noticeable loss of sharp edges and fine image details due to the smoothing effect of the blur kernel. After adding Gaussian noise, the image quality degrades further.

Applying the pseudoinverse of the blur matrix successfully reconstructs many of the original image features, demonstrating that the inverse problem can be solved using the linear matrix formulation. However, the reconstructed image is not identical to the original. Some residual blur and amplified noise remain because the pseudoinverse is highly sensitive to noise, especially when the blur matrix is ill-conditioned. Consequently, small perturbations in the observed image are magnified during inversion, limiting the reconstruction quality.

Overall, the experiment demonstrates that image blurring can be modeled as a linear matrix operation and that image restoration can be performed by solving a linear inverse problem using the Moore–Penrose pseudoinverse.

Q4:

```
f = double(f(:,:,1));
f = (f - min(min(f)))/(max(max(f))-min(min(f)));
f_d = downsample(f, 4);
f_d = downsample(f_d', 4)';

K = zeros(size(f_d));
h = Gaussian(sigma, [3,3]);
h = h/sum(h, 'all')

K(1:3, 1:3) = h;
D = zeros(size(K).^2);

for r = 1:size(K, 1)
    Kp = circshift(K,r-1,1);
    for c = 1:size(K, 1)
        Kz = circshift(Kp,c-1,2);
        D((r-1) * length(K) + c, :) = reshape(Kz, [1, length(K).^2]);
    end
end

g = conv2(f_d, h, 'same');

g_noisy = g + randn(dims) * 0.05;

g_resh = reshape(g_noisy, [size(g, 1) * size(g, 2), 1]);
f_reconst = zeros(size(g_resh));

for i = 1:num
    f_reconst = f_reconst + beta * D' * (g_resh - D*f_reconst);

    err = norm(g_resh-D*f_reconst);
    fprintf('iter= %d, error= %d \n', i, err)
    if err < eps
        break
    end
end
```

This code implements the **Landweber iterative method** to recover a clear image from a blurred and noisy version. The problem is expressed as:

$$\mathbf{g} = \mathbf{H}\mathbf{f} + \mathbf{n}$$

where $\mathbf{f}$ is the original image, $\mathbf{H}$ is the blur operator, $\mathbf{n}$ is noise, and $\mathbf{g}$ is the observed image.

Image Preparation

The code loads an image, converts it to grayscale, normalizes the pixel values to the range **[0, 1]**, and downsamples it by a factor of **4** to reduce computational cost.

Blur Kernel

A Gaussian blur kernel with a standard deviation $\sigma = 0.4$ is created. This kernel represents how the image becomes blurred. The kernel size is 3×3 and is normalized so that the sum of all its elements equals 1.

Blur Matrix Construction

The blur kernel is converted into a large matrix $\mathbf{D}$ using circular shifts. This allows the convolution operation to be represented as the matrix multiplication:

$$\mathbf{g} = \mathbf{D}\mathbf{f}$$

Blur and Noise Addition

The code applies the blur to the original image using convolution and then adds Gaussian noise with a standard deviation of **0.05** to simulate real-world imaging conditions.

Landweber Iteration

The algorithm iteratively updates the image estimate using the equation:

$$\mathbf{f}_{\text{new}} = \mathbf{f}_{\text{old}} + \beta \mathbf{D}^T (\mathbf{g} - \mathbf{D}\mathbf{f}_{\text{old}})$$

The term $(\mathbf{g} - \mathbf{D}\mathbf{f}_{\text{old}})$ is the residual error between the observed image and the blurred estimate. The transpose matrix $\mathbf{D}^T$ projects this error back into the image domain. The parameter β is the step size that controls how much correction is applied during each iteration.

Stopping Criteria

The iteration stops when the error norm falls below $\epsilon = 0.01$ or when the maximum number of **1000 iterations** is reached.

Parameter Significance

- σ (**sigma**) controls the amount of blur.
- β (**beta**) controls the convergence speed and stability.
- ϵ (**epsilon**) determines the reconstruction accuracy.
- The **noise level** affects how difficult the image recovery becomes.

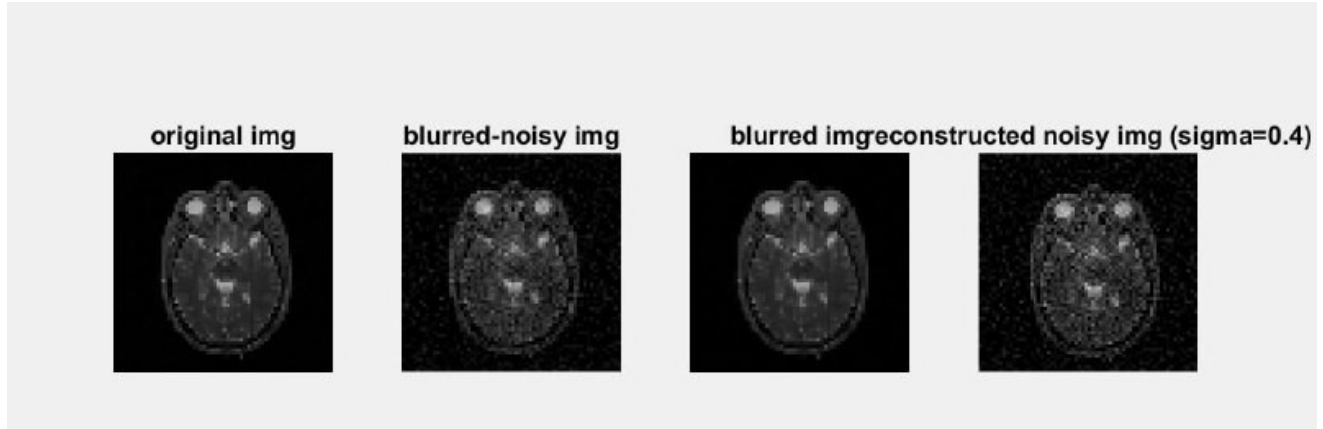
Results Display

The code displays four images:

- Original image
- Blurred image

- Blurred image with added noise
- Reconstructed image

This allows a visual comparison of the restoration quality.



Mathematical Principle

The Landweber method performs **gradient descent** on the least-squares cost function:

$$J(f) = \|Df - g\|^2$$

At each iteration, the estimate moves in the direction that reduces this error. The method is stable because it only requires multiplication by the transpose of **D** and does not require computing the inverse of the blur matrix.

Conclusion

The Landweber iterative method provides a simple and effective approach for solving the image deconvolution problem. By repeatedly correcting the image estimate using the residual error, the algorithm gradually recovers the original image from blurred and noisy observations.

Q5:

Anisotropic Diffusion Filtering in Medical Image Processing

Introduction

Anisotropic diffusion filtering is a technique used to reduce noise in images while preserving important details such as edges, lines, and structural features. Unlike traditional linear filtering methods, which tend to blur edges and fine details, anisotropic diffusion selectively smooths homogeneous regions while maintaining important image boundaries. This method is widely used in medical image processing because preserving anatomical structures is essential for accurate diagnosis.

The process is based on a diffusion equation whose behavior adapts according to the local image content. Greater smoothing is applied in uniform regions, while diffusion is reduced near edges and other significant structures. As a result, image noise is reduced without sacrificing important structural information.

Mathematical Foundation

The general diffusion equation is

$$\partial I / \partial t = \nabla \cdot (c(x, y, t) \nabla I)$$

where:

- I is the image intensity,
- $c(x, y, t)$ is the diffusion coefficient, which varies with image location and time.

Implementation Steps

The code first initializes the diffusion parameters, including the time step, total diffusion time, and diffusion scaling factor. During each iteration, it computes the image gradient, gradient magnitude, and divergence.

An edge-stopping function determines the diffusion coefficient, allowing either simple smoothing or a more advanced edge-preserving model. The image is updated after each iteration to simulate the diffusion process. Intermediate results are periodically displayed to compare the original and filtered images. The iterations continue until the specified final time is reached.

Code Analysis

The file **S2_Q5_Anisotropic_Diffusion.m** implements the anisotropic diffusion algorithm step by step.

First, the required parameters are initialized. During each iteration, the image gradient is computed to detect edges and structural information. The gradient magnitude determines the amount of diffusion that should occur at each pixel. The diffusion coefficient is then calculated using an edge-stopping function, ensuring that diffusion is reduced near edges. Finally, the divergence of the diffusion flow is computed to update the image.

As the iterations proceed, the image becomes progressively smoother in homogeneous regions while sharp edges and important structures remain well preserved.

Results

The output demonstrates that anisotropic diffusion effectively removes noise while preserving significant image features. Compared with conventional smoothing filters, this method produces cleaner images without noticeably blurring object boundaries. This property makes anisotropic diffusion particularly valuable in medical imaging applications, where maintaining diagnostic details is essential.

